Tools, Not Prompts: Revisiting LLM-Based Legacy PHP Modernization with an Agent Harness

Ianiv Schweber - Just a random guy on the Internet

November 2, 2026

Abstract

A recent study evaluated five large language models (LLMs) on modernizing 100 WordPress 4.0 PHP files to PHP 8.3. It concluded that every model ended with fewer behaviorally compatible files than leaving the code unchanged (83 of 100). In that study each model received one prompt per file, with no tools, tests, or runtime feedback. We re-run the study's own released harness (630 hidden PHPUnit checks and a Rector-based legacy-pattern metric) with Claude Opus 5.5 under two conditions. In the original single-shot condition, using the study's exact prompt and output extractor, Opus 5.5 produces 94 compatible files with one regression across 630 checks. In an agent condition, Opus 5.5 runs as a coding agent with PHP 8.3, the file's checks, a linter, and Rector, and outputs are scored against a pristine copy of the harness. It produces 99 compatible files with zero regressions, repairs 93 of the 99 checks the legacy code fails, and resolves 92.4% of Rector's 511 PHP-version findings. The one remaining file cannot pass under the single-file protocol, because its checks load an unmigrated dependency that PHP 8 rejects. We first validate our harness by reproducing the original no-change baseline and the published Sonnet 4 results exactly. We conclude that the original negative result reflects the evaluation condition and model generation rather than a limit of LLM-based migration. Studies of LLMs for software maintenance should report an execution-feedback condition alongside single-shot generation.

1 Introduction

Much of the web still runs PHP that was written for PHP 5. PHP 7 and 8 removed or redefined many of the constructs that code depends on: `each()`, `create_function()`, curly-brace string offsets, PHP 4-style constructors, and lenient handling of `null` and non-countable values. Moving such code to a supported runtime is costly, repetitive, and easy to get subtly wrong, which makes it a natural target for automation.

The study we revisit [1] asks whether LLMs can do this work safely. It samples 100 files from WordPress 4.0 and asks five models to migrate each file to PHP 8.3 in a single prompt. It then scores the outputs with 630 hidden PHPUnit checks and with Rector's PHP-version rules. Its headline finding is negative. Under PHP 8.3, 83 of the 100 unmodified files are behaviorally compatible, while the best model reaches 59. The rule-based tool Rector reaches 82.

That finding is accurate for the condition the study measures, but it is easy to read more broadly than it should be. In the study, a model sees one file and nothing else, and it never runs, lints, or tests its output. Practitioners do not use LLMs this way for migrations. Coding agents iterate against an interpreter and a test suite. The models evaluated in [1] were released between December 2024 (Llama 3.3 70B [2]) and September 2025 (GPT-5 Codex [3]), 13 to 23 months before the study was submitted in October 2026. Claude Sonnet 4 dates from May 2025 [4], and both Gemini 2.5 models became generally available in June 2025 [5]. Newer model generations have been released

since, and we evaluate one of them. Two questions therefore remain open: how much of the gap comes from the model generation, and how much comes from the absence of execution feedback.

We answer them with the study's own artifacts. We pose three research questions:

- **RQ1 (Reproduction).** Can the released harness be run independently and reproduce the published baseline and model results?
- **RQ2 (Model).** Under the identical single-shot protocol, how does a current frontier model (Claude Opus 5.5) compare with the models in the original study and with the unchanged code?
- **RQ3 (Harness).** When the same model runs as a coding agent with an interpreter, the file's checks, a linter, and Rector, how close does it come to a complete and behavior-preserving migration?

Our contributions are:

1. An independent reproduction of the study's evaluation on macOS with PHP 8.3 in a Linux VM. It matches the no-change baseline check for check and the published Sonnet 4 results exactly, and it identifies a one-check discrepancy between the paper's text and its package.
2. A single-shot replication with Opus 5.5 that keeps the study's prompt and extractor. Opus 5.5 alone exceeds the no-change baseline (94 vs. 83 compatible files).
3. An agent harness, with tamper-proof scoring, that reaches 99 of 100 compatible files with zero regressions. We show that the remaining file is not passable under the single-file protocol.
4. A released replication package with all outputs, scores, and each agent session's final report and run statistics.

2 Background and Related Work

Rule-based migration. Rector [6] rewrites PHP abstract syntax trees using rule sets keyed to language versions, such as `UP_TO_PHP_83`. Its rules are local and mechanical, so they rarely change behavior that already works. For the same reason, they cannot repair failures that depend on semantics: a `count()` on `null`, which now throws, or a comparison whose result changed in PHP 8. The original study uses Rector both as a baseline and as the source of its legacy-pattern metric.

Evaluating behavior, not text. TransCoder [7], an unsupervised neural translator between C++, Java, and Python, introduced *computational accuracy*: a translation counts as correct only if it produces the reference outputs on unit tests. Its motivation is the gap that the hidden checks of [1] also address. Only 3.1% of TransCoder's C++-to-Java translations matched the reference text, yet 60.9% passed the tests. TransCoder also used the tests to choose among beam-search candidates, which raised computational accuracy by up to 33.7 points (Java to Python). Choosing by execution is the simplest form of the feedback our agent condition provides.

LLMs silently change behavior when translating. Pan et al. [8] found that 2.1% to 47.3% of LLM code translations were correct, depending on the model. Most failures did not compile, but 13.4% compiled and ran yet failed tests. That silent change of behavior is the failure mode the hidden checks of [1] detect. Feeding error output and failing test cases back to the model raised GPT-4's success rate by 12.33% after one round, although some repairs introduced new bugs. Our preservation metric tracks this risk: repair in a loop must not cost regressions.

Execution feedback, not iteration, drives the gain. Self-debugging [9] lets a model execute its code and revise it over up to 10 rounds. On C++-to-Python translation it raised Codex from 80.4% to 92.5% when the model could see unit-test results, but only to 83.9% when it could reflect on its code without executing it. This split between generation and execution feedback mirrors our RQ2 and RQ3 at function scale.

Agents on repository-scale tasks. SWE-bench [10] judges a patch by tests that must go from failing to passing and tests that must keep passing. These are the same repair and preservation criteria that [1] uses. Its baselines generate a patch in one pass from retrieved files, and the best resolved 1.96% of issues. With the same model (GPT-4 Turbo), SWE-agent [11] raised the resolve rate on SWE-bench Lite from 2.67% for that single-pass baseline to 18.0%, by letting the model navigate the repository, edit files, and run code. Its ablations credit a linter that rejects syntactically invalid edits with 3.0 of those points. Our agent's `tools/lint` serves the same role.

Evaluation condition as a variable. Our study treats the presence of execution feedback as an experimental factor, separate from the model. To isolate the model's contribution we hold the prompt, file, and scoring fixed (RQ2). We then add the tool loop with the model held fixed (RQ3).

3 The Original Study

Dataset. The study [1] draws on WordPress 4.0's 482 PHP files. It keeps the 397 that have Rector findings and selects 100 by a deterministic, stratified procedure that covers all rule types. By size, 31 files have at most 200 lines, 31 have 201–500, 26 have 501–1,000, and 12 exceed 1,000 lines. Together they total 54,321 lines.

Hidden checks. The study defines 630 PHPUnit checks: 109 reachability checks, 441 output or state checks, and 80 semantic-fingerprint checks. Each test file targets one subject. It runs the subject in a sandbox built from the WordPress 4.0 tree plus per-file stubs, under PHP 8.3 with `E_ALL`, and it counts deprecation notices as failures. Unmodified, the files pass 531 checks. The harness report in the study's replication package [12] attributes the 99 failures to removed or changed PHP features: 59 parse errors, 21 output mismatches, 8 undefined symbols, 6 type or signature errors, and 5 fatal runtime errors.

Metrics. For a migrated variant, each check falls into one of four transitions relative to the original: preserved pass, regression, repair, or unrepaired failure. The study reports:

- **Compatible files:** files whose checks all pass.
- **Preservation:** preserved passes ÷ 531, the original passes that still pass.
- **Repair:** repairs ÷ 99, the original failures that now pass.
- **Legacy-pattern resolution:** Rector `Php*` rule detections in the original (511 across 44 rule types) that are no longer detected after migration. Rector runs on each file in isolation, and unparseable output resolves nothing.

Condition. Each model receives a fixed prompt that asks a senior PHP developer to migrate the file to PHP 8.3 while preserving functional equivalence, followed by the full file. The settings are temperature 0.2 and at most 64,000 output tokens. The study keeps the first complete response and extracts code between `// MIGRATION_START` and `// MIGRATION_END` markers. The model sees no other files, tests, or errors.

Reported results. Every LLM falls below the unchanged code on compatible files: Sonnet 4 reaches 59, GPT-5 Codex 58, Gemini 2.5 Pro 55, Gemini 2.5 Flash 50, and Llama 3.3 70B 33, against Rector's 82. The full figures appear alongside ours in Table 2.

4 Methodology

4.1 Environment and harness reproduction

We download the study's replication package [12] and run it unchanged except for the runner. The package's PowerShell runner sets three environment variables and calls PHPUnit, so we port it to a 10-line bash script. PHP runs in a Linux VM under Apple's `container` (v1.6.1, Virtualization.framework) from the official `php:8.3-cli` image, which provides PHP 8.3.36. It includes the bcmath, curl, dom, mbstring, pdo, simplexml, tokenizer, xml, and zip extensions, with `error_reporting=E_ALL`. Tools come from the package's lock files: PHPUnit 9.6.34 and Rector 2.1.0.

We validate the reproduction three ways before running any model (RQ1):

1. Run the unchanged originals and diff the per-check testdox output against the package's `OriginalTestResults.txt`.
2. Re-score the package's archived Sonnet 4 outputs end to end and compare with the published summary.
3. Run Rector on every original and count `Php*` findings against the package's metadata.

4.2 Single-shot condition (RQ2)

We reuse the study's prompt template verbatim, insert the full file, and extract code with a line-for-line port of its extractor. Like the original, we read files as UTF-8 with undecodable bytes ignored. Calls go to Claude Opus 5.5 through the Claude Code CLI in print mode, with every tool disabled, a one-line system prompt, no user or project settings, and a 64,000-token output cap. As in [1], we keep the first complete response and regenerate incomplete ones, up to three attempts.

Two details differ from a raw API call. The CLI does not expose temperature, so sampling uses the default. Opus 5.5 also uses adaptive extended thinking by default, which we leave on: on file 004 it spent 3,786 thinking tokens. One response (file 014, 136 KB of code) exceeded the per-turn output cap and was continued across two assistant turns. We join the text of all turns, which is equivalent to a single response under the study's 64,000-token cap.

4.3 Agent condition (RQ3)

Each file gets its own workspace. It is a copy-on-write clone of the harness with every archived model output, result file, and report removed, so the agent cannot see how other models did. The subject directory starts as a copy of the originals. The agent may edit exactly one file, the migrated copy of its subject.

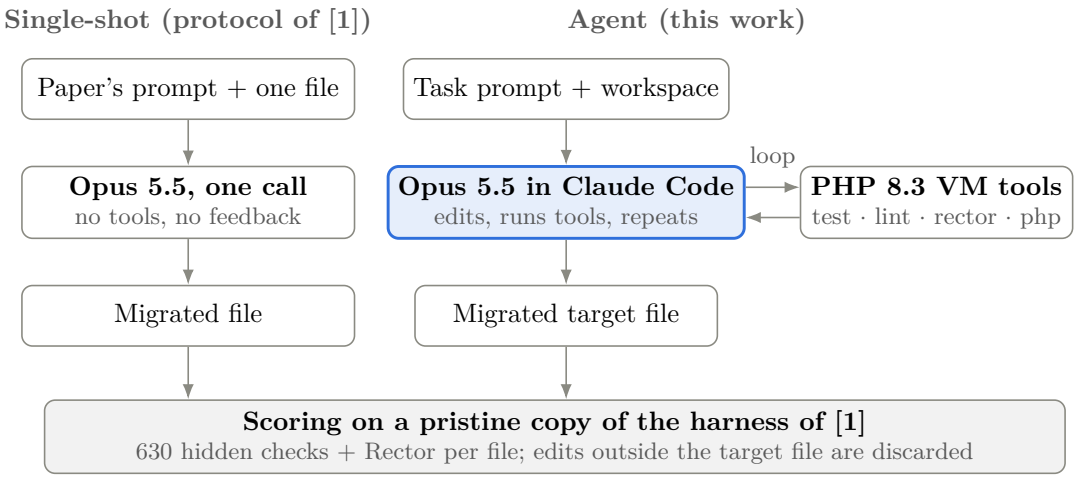


Figure 1: The two experimental conditions and the shared scoring pipeline. Both receive the same file and are scored by the same pipeline; only the agent can run code against the checks before submitting.

Five tools run inside the PHP 8.3 VM:

- `tools/test` runs the file's checks against the migrated copy.
- `tools/test-original` runs the same checks against the original.
- `tools/lint` runs `php -l`.
- `tools/rector` lists the remaining `Php*` findings, with the diff Rector would apply.
- `tools/php` runs the PHP CLI for scratch scripts.

The agent can read the checks, the stubs, and the full WordPress 4.0 tree. Its instructions restate the study's goal of migrating to PHP 8.3 while preserving observable behavior. They add a definition of done: every check passes, no Rector finding remains unless applying it would change behavior or call compatibility (with a reason for each one kept), and lint is clean. They forbid edits outside the target, special-casing test inputs, detecting the test environment, and deleting uncovered functionality.

The agent is Claude Code (v2.1.316) in headless mode with model `claude-opus-5-5` and default effort. It runs in `dontAsk` permission mode, with an allowlist of file tools, the five tools, and read-only shell commands. Web access, `curl`, and `wget` are denied, and user settings, hooks, and MCP servers are not loaded. Each file gets one session with no turn limit, and six sessions run in parallel.

4.4 Scoring

Both conditions are scored by the same pipeline. It builds a fresh copy of the released harness, copies in only each run's migrated target files, and runs all checks. It then runs Rector on each migrated file in its own directory, matching the study's single-file isolation. Because tests, stubs, and configuration come from the pristine copy, an agent's edits outside its target cannot affect its score. After each session we also diff the agent's workspace against the pristine harness and record any changes outside the target. We count the 630 checks of the study and exclude the package's setup smoke test (Section 5.1).

Table 1: Reproduction checks against the released package.

Check	Published	Reproduced
Originals: per-check testdox outcome	`OriginalTestResults.txt`	identical, 631 of 631 lines
Originals: checks passed	531 of 630 (paper text)	532 of 631 (package); 531 of 630 without the smoke test
Originals: compatible files	83	83
Sonnet 4 archived outputs: per-check testdox	package `testdox.txt`	identical
Sonnet 4: compatible / preservation / repair	59 / 76.3% / 68.7%	59 / 76.3% / 68.7%
Sonnet 4: Rector findings resolved	272	272
Rector `Php*` findings on originals	511	511 (per-file isolation); 574 (whole directory)

Table 2: Results on the 100 files, 630 checks, and 511 Rector findings. Rows above the rule are from [1]. The paper does not publish findings introduced for most rows, so those cells are marked –; the Sonnet 4 figure comes from our re-scoring of its archived outputs.

Approach	Compatible files	Checks passed	Regressions	Repairs	Rector findings resolved	Rector findings introduced
Unchanged originals	83	531	–	–	0 (0%)	–
Llama 3.3 70B	33	337	215	21	156 (30.5%)	–
Gemini 2.5 Flash	50	393	198	60	273 (53.4%)	–
Gemini 2.5 Pro	55	446	147	62	372 (72.8%)	–
GPT-5 Codex	58	467	122	58	297 (58.1%)	–
Claude Sonnet 4	59	473	126	68	272 (53.2%)	17
Rector 2.1.0	82	581	12	62	506 (99.0%)	–
Opus 5.5, single-shot	**94**	**617**	**1**	**87**	**345 (67.5%)**	27
Opus 5.5, agent	**99**	**624**	**0**	**93**	**472 (92.4%)**	4

5 Results

5.1 RQ1: The harness reproduces the published results

The reproduction matches the study wherever both can be compared (Table 1). The only discrepancy is a one-check counting difference in the paper's text.

The package contains 631 checks, and both its results file and our run pass 532 of them. The extra check is a setup smoke test, and excluding it gives the paper's 630 and 531. Rector's count depends on context. Run over the whole directory, Rector reports 61 extra `RemoveExtraParametersRector` and 7 extra `AddOverrideAttributeToOverriddenMethodsRector` findings, because it can resolve signatures defined in other files. It also misses 5 that it finds in isolation. Only per-file isolation reproduces 511, so we use it throughout.

5.2 Main results

Figure 2 recreates the file-level view of [1] and adds our two conditions. Every model in [1] breaks between 32 and 51 of the 83 working files. Opus 5.5 breaks one file single-shot and none as an agent, and the agent fixes 16 of the 17 broken originals. The seventeenth is file 009 (Section 5.5).

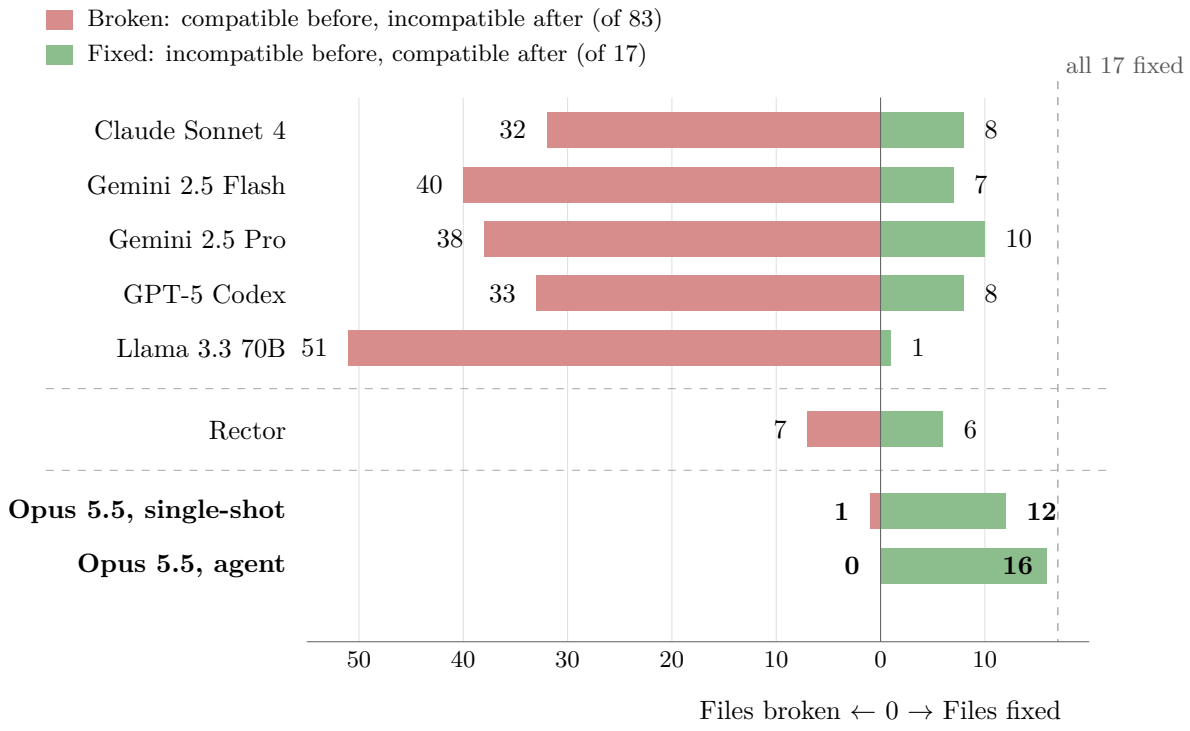


Figure 2: Files broken and fixed per approach. The top five rows are the models of [1], then Rector, then our two conditions. Rows from [1] are recomputed from the package's `per_file.csv` and match the paper's figure; Opus rows come from `runs/*/score/per_file.csv`.

5.3 RQ2: The model alone exceeds the unchanged code

With the study's prompt and no feedback, Opus 5.5 produces 94 compatible files. That is 11 more than the unchanged code, 12 more than Rector, and 35 more than the best model in [1]. It breaks one check across 531 original passes, a semantic-fingerprint check for nav-menu sorting in file 054. It also repairs 87 of the 99 failing checks. All 100 responses were complete on the first attempt.

The single-shot outputs fail checks in 6 files. Apart from 009 (Section 5.5), the failures are narrow: 3 of 7 checks for `class-wp.php` request parsing (007), one AC3 lookup table (016), POP3's empty-mailbox marker (021), JSON encoding of primitives (030), and the nav-menu regression (054). On legacy patterns, single-shot Opus 5.5 resolves 67.5% of findings and introduces 27. That is more than Sonnet 4 and GPT-5 Codex, slightly less than Gemini 2.5 Pro, and well below Rector's 99%. Only 21 files leave both all checks passing and no Rector findings.

5.4 RQ3: With tools, the migration is complete and behavior-preserving

As an agent, Opus 5.5 breaks none of the 531 passing checks and repairs 93 of 99 failing ones. All 6 unrepaired checks belong to file 009. It fixes all five files the single-shot run left failing and raises resolution from 67.5% to 92.4%, introducing 4 new findings. Altogether 68 files finish with all checks passing and no remaining Rector findings.

The agent's instructions allow a finding to stay when applying it would change behavior or call compatibility. Of the 511 original findings, 39 remain, and the agent introduced 4 new ones. The resulting 43 detections fall under 10 rules, most of them optional rewrites (Figure 3).

In 30 of the 32 files that keep findings, the agent's final report names every kept rule and gives a

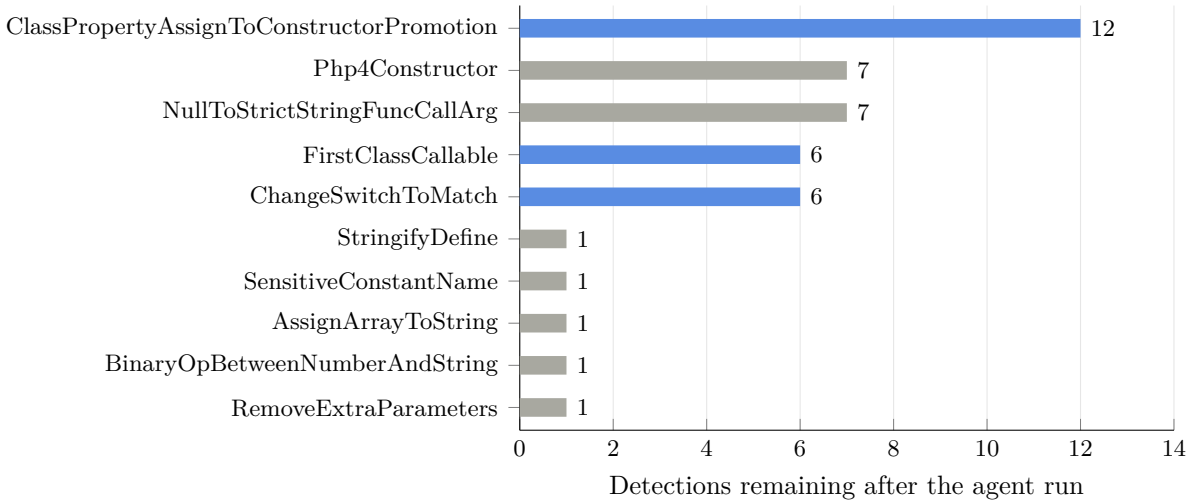


Figure 3: Rector `Php*` detections in the agent run's 100 migrated files, by rule: 43 detections across 10 rules, of which 24 are optional rewrites (blue). Grey: old-style constructors kept as compatibility wrappers (`Php4Constructor`), casts on request values (`NullToStrict`), and single cases. Source: `runs/opus55/score/per_file.csv`.

reason. The reports for 010 and 022 name some but not all. The reasons fall into three groups:

- **Optional modernization.** Constructor promotion, `switch` to `match`, and first-class callables are valid PHP 8.3 without them and raise no notice. Agents declined them where they would reorder properties visible to `get_object_vars()`, or where `match`'s strict comparison would differ from `switch`'s loose one.
- **Compatibility shims.** Seven `Php4ConstructorRector` findings remain where the agent added `__construct` but kept the old same-name constructor as a forwarding wrapper. WordPress subclasses such as `wp_xmlrpc_server` still call `$this->IXR_Server(...)`.
- **Defensive casts.** The agents declined `NullToStrictStringFuncCallArgRector` casts on request superglobals whose values are always strings at runtime.

No agent edited a file outside its target. The tamper log is empty for all 100 sessions. All 100 sessions hit at least one permission denial, 123 in total. Most were compound shell commands for bulk edits (`cd ... && perl -pi ...`, 81 cases) that fell outside the allowlist. Despite the denials, every session ended with a migrated file and a final report.

Cost and effort. The agent run cost $39.66 in total: a median of $0.30, 14 turns, and 73 s per file, with at most $1.56, 59 turns, and 359 s. The single-shot run cost $26.92, a median of $0.19 per file. At six sessions in parallel, the batch of 99 sessions took about 30 minutes of wall-clock time.

5.5 File 009 cannot pass under the single-file protocol

Every approach in Table 2 scores 0 of 6 on `getid3.php` (009). Its checks construct a `getID3` object, whose constructor `includes` `getid3.lib.php` from the sandboxed WordPress tree. That library file is not the subject being migrated. It still uses curly-brace string offsets, which PHP 8 rejects at compile time with an uncatchable fatal error. No edit confined to `getid3.php` can prevent the include from failing. The agent for 009 diagnosed this in its report and stopped rather than edit files

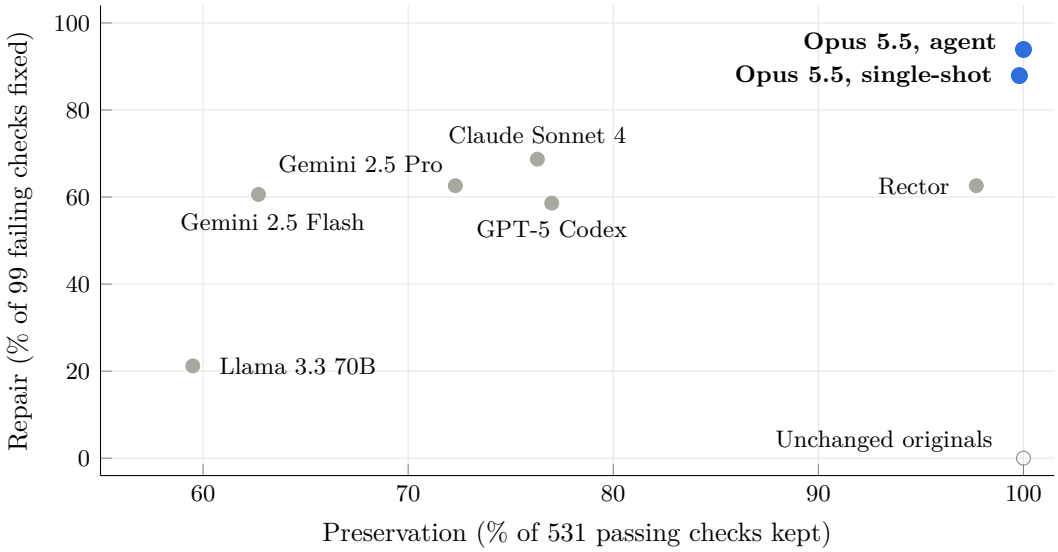


Figure 4: Preservation vs. repair per approach. The models in [1] trade one rate against the other in the lower left; Rector sits at high preservation but repairs no more than they do; both Opus conditions occupy the corner that neither rules nor older models reach. Rows from [1] are computed from the package's `summary.csv`; Opus rows from `runs/*/score/summary.json`.

it was told not to touch. Excluding 009, the agent condition passes every check on all 99 remaining files.

6 Discussion

Model and harness contribute different things. Holding the protocol fixed, the change of model alone moves compatible files from 59 to 94 and regressions from 126 to 1. Holding the model fixed, adding the tool loop moves compatible files from 94 to 99 and Rector resolution from 67.5% to 92.4%. Most of the behavioral gap reported in [1] therefore closes with a newer model. Prior work found the same split between generating and executing. In self-debugging [9], execution feedback added about 12 points on translation and reflection without execution about 3. In SWE-agent [11], the same model resolved roughly seven times more issues once it could act on a repository. The agent loop contributes the long tail of behavioral fixes and, above all, completeness of the modernization itself. A single-shot model leaves about a third of the legacy patterns in place, while the agent removes most of them and explains the rest.

The agent dominates Rector on both axes. Rector is the safest single-shot option in [1]: 97.7% preservation, because its rewrites are mechanical. Its 62.6% repair shows the limit of local rules. Many legacy failures are semantic, for example `count()` on `null`, removed functions that need a behavior-equivalent replacement, and `htmlspecialchars` defaults that changed in PHP 8.1. Repairing them requires reasoning about intended behavior. The agent reaches 100% preservation and 93.9% repair, and it uses Rector as a tool. Rector's diff shows the agent what remains, and the checks tell it which suggested rewrites are unsafe. Rule-based and LLM-based migration are complements in this workflow, not rivals.

Figure 4 places each approach by the two rates. Listing 1 shows two repairs from file 004 (`class-IXR.php`) that rules cannot make. Rector's `Php4ConstructorRector` renames `IXR_Server()`

Listing 1: File 004: original, Rector's rewrite, and the Opus 5.5 agent's output (excerpts).

```
// Original (WordPress 4.0)
function IXR_Server($callbacks = false, $data = false, $wait = false) { ... }
...
if (count($args) == 1) {          // TypeError in PHP 8 when $args is null

// Rector UP_TO_PHP_83: renames the constructor, drops IXR_Server(), leaves count() as is
function __construct($callbacks = false, $data = false, $wait = false) { ... }

// Opus 5.5 agent
public function __construct($callbacks = false, $data = false, $wait = false) { ... }

/**
 * PHP4-style constructor, kept for backward compatibility
 * (e.g. wp_xmlrpc_server calls $this->IXR_Server()).
 */
public function IXR_Server($callbacks = false, $data = false, $wait = false)
{
    self::__construct($callbacks, $data, $wait);
}
...
// (PHP < 8 semantics of count(): null counts as 0, scalars as 1)
$argCount = $args === null ? 0 : (is_countable($args) ? count($args) : 1);
if ($argCount == 1) {
```

to `__construct()` and drops the old name. But WordPress's own `wp_xmlrpc_server` still calls `$this->IXR_Server($this->methods)` (`class-wp-xmlrpc-server.php`, line 159). In PHP 8 a same-name method no longer acts as a constructor. Rector's rename restores construction but makes that explicit call fail with an undefined-method error. Rector also has no rule for `count()` on `null`, which throws a `TypeError` in PHP 8. According to its final report, the agent found both by reading the callers and running the checks.

The kept wrapper is why `Php4ConstructorRector` still reports findings after the agent run (Figure 3). The agent chose a remaining Rector finding over a broken caller.

Single-shot results are a lower bound. The original condition measures how often a model can rewrite a file correctly with no ability to run it. That is a meaningful quantity: it bounds what a chat-window workflow delivers, and it shows models are not reliable when they cannot observe behavior. It does not measure what LLM-based migration can achieve, and the conclusion of [1] reads more broadly than its condition supports. We suggest that studies of LLMs for maintenance tasks report at least two conditions, single-shot and with execution feedback, and name the model generation in their conclusions.

Harness details matter at this level of performance. Once a system passes nearly every check, artifacts in the harness dominate the remaining error. File 009 accounts for every check the agent fails. It is failed by every approach, and no change to the subject can fix it. The Rector metric also depends on running each file in isolation: a directory-level run inflates the original count by 12%. Benchmarks for migration should check that each subject is passable under their own protocol, for example by including an oracle migration.

Practical implications. At a median of $0.30 and about a minute per file, an agent loop over an existing test suite is an inexpensive first pass for legacy PHP migration. Two caveats matter. The checks here are a sample of each file's contract, so passing them does not prove equivalence, and human review of the diffs remains necessary. Where no tests exist, an agent can still lint, run Rector, diff behavior against the original on PHP 7, or write its own tests. Reflexion [13] reached 91% pass@1 on HumanEval code generation with up to six unit tests that the model generated itself. That condition is weaker than ours, and we do not measure it.

7 Threats to Validity

Test visibility (construct). In the agent condition the model can read and run the same checks that score it. Its results show what a developer with a test suite achieves, not how a migration generalizes to behavior no test covers. Several measures reduce overfitting: instructions that forbid special-casing and environment detection, scoring against a pristine harness, and the Rector metric, which no test controls. They do not remove it. A held-out split of checks would separate the two effects, and we leave that to future work. The single-shot condition has no such exposure and still reaches 94 files.

Single run, uncontrolled sampling (conclusion). Each condition was run once per file. The CLI does not expose temperature, so we could not use the study's 0.2. Results may vary between runs, especially for the six single-shot failures. The margins over the original models (35 or more compatible files, and over 100 fewer regressions) are far larger than plausible run-to-run variation, but the five-file gap between our two conditions is not.

Thinking and interface (internal). Opus 5.5 used adaptive extended thinking in both conditions. The client code in [12] sets no thinking or reasoning parameters, so each model in [1] ran with its provider's default. For Sonnet 4 that means no extended thinking, while both Gemini 2.5 models think by default [14]. Our single-shot calls went through the Claude Code CLI with a one-line system prompt, not a raw API call with none. We consider both part of how the model is used today, but they make the single-shot comparison less than identical.

Environment (internal). We used PHP 8.3.36 rather than 8.3.22, and Linux on Apple's Virtualization.framework rather than Windows. The originals and the archived Sonnet 4 outputs reproduce check for check, so neither difference affected those scores. We cannot rule out effects on the new outputs.

Metrics (construct). Hidden checks are a sample of each file's contract. Rector `Php*` findings are a proxy for modernization that rewards optional style changes. A low resolution rate can reflect a deliberate choice not to change behavior, as Section 5.4 shows.

Generalization (external). The 100 files are a purposive sample from one application at one version, as [1] notes. We inherit that limit and add one model from one vendor.

Agent process not recorded (internal). The agent runs used Claude Code's headless JSON output, which records each session's final report and statistics (turns, tokens, cost, permission denials) but not the turn-by-turn conversation. Statements about how an agent reached a result

therefore rest on its own final report. The results themselves do not: every outcome is verified independently by scoring the submitted file.

Evaluator independence. The agent harness, prompts, and analysis were built with the help of a Claude-based coding assistant, from the same vendor as the model evaluated. The scoring code is the original study's harness, and all outputs, final reports, and run statistics are released for inspection.

8 Conclusion

The study we revisit found that LLMs leave legacy PHP less working than they found it. Re-running its own harness, we find that this no longer holds for a current model, and that it never held for a model with tools. Under the identical single-shot protocol, Claude Opus 5.5 produces 94 compatible files against 83 for the unchanged code. As a coding agent with an interpreter, the file's checks, and Rector, it produces 99 with no regressions and resolves 92.4% of legacy-pattern findings. The remaining file fails because of the benchmark's single-file protocol, not the migration. The question for LLM-based modernization is no longer whether a model can rewrite a file blind, but how to give it the feedback a developer would have, and how to evaluate it under that condition.

Data availability. The original replication package is on Zenodo [12]. Our package contains the bash port of the runner, the workspace and tool scripts, the agent prompt, the single-shot runner, the scoring and metrics code, each agent session's final report and run statistics, all single-shot responses, the migrated files from both conditions, and per-file and summary scores. It is public at `https://github.com/ianiv/php-modernization-agent-replication`. Local paths in the released records are replaced with `<repo>`; the records are otherwise unedited.

References

[1] Afridi Mahmud, Abid Hasan Khondaker, Sadif Ahmed, Md Nafiu Rahman, and Rifat Shahriyar. When modernization breaks working code: A single-file study of LLMs on legacy PHP. arXiv:2610.40269, 2026. URL `https://arxiv.org/abs/2610.40269`.

[2] Meta. Llama 3.3 70b instruct model card, 2024. URL `https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct`. Release date: December 6, 2024.

[3] OpenAI. Introducing upgrades to Codex, 2025. URL `https://openai.com/index/introducing-upgrades-to-codex/`. September 15, 2025; release of GPT-5-Codex.

[4] Anthropic. Introducing Claude 4, 2025. URL `https://www.anthropic.com/news/claude-4`. May 22, 2025.

[5] Google. We're expanding our Gemini 2.5 family of models, 2025. URL `https://blog.google/products/gemini/gemini-2-5-model-family-expands/`. June 17, 2025; 2.5 Pro and Flash generally available.

[6] Rector: Automated PHP upgrades and refactoring, v2.1.0. URL `https://github.com/rectorphp/rector`. Built on nikic/php-parser; `UP_TO_PHP_83` level set.

[7] Baptiste Rozière, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html.

[8] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024. URL https://arxiv.org/abs/2308.03109.

[9] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2304.05128.

[10] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

[11] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

[12] Replication package for "When Modernization Breaks Working Code" (anonymized). Zenodo record 20274676, 2026. URL https://zenodo.org/records/20274676.

[13] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract.html.

[14] Google. Gemini thinking: Model defaults. URL https://aistudio.google.com/docs/thinking. Accessed 2026-11-02.